

ND6

Remember, changes to question templates won't automatically update quizzes that are already using those questions.

☐ Show Question Details



Question 2 pts



Phát biểu nào đúng nhất về “function specification” (đặc tả hàm)?

- ☐ Chỉ cần code chạy được là đủ
- ☐ Đặc tả mô tả hàm nhận gì, trả gì, và điều kiện/ý nghĩa kết quả
- ☐ Đặc tả chỉ là phần comment cho đẹp
- ☐ Đặc tả thay thế hoàn toàn việc test

[move/copy question to another bank](#)



Question 2 pts



Bạn viết hàm:

```
def avg(a, b):
```

```
    return (a + b) / 2
```

Test nào sau đây bắt lỗi tốt nhất nếu ai đó lỡ sửa nhầm thành $(a+b)/3$?

- ☐ `assert avg(2, 4) == 2`
- ☐ `assert avg(2, 4) == 3`
- ☐ `assert avg(0, 0) == 0`
- ☐ `assert avg(-1, 1) == 0`

[move/copy question to another bank](#)



Question 2 pts



Xét đoạn mã sau:

```
x = 10
```

```
def my_func():
```

```
    x = 5
```

```
my_func()
```

```
print(x)
```

Giá trị in ra là bao nhiêu và tại sao?

- ☐ Lỗi UnboundLocalError
- ☐ 10, vì việc gán x=5 bên trong hàm tạo ra một biến cục bộ mới
- ☐ 15, vì Python cộng dồn các giá trị trùng tên
- ☐ 5, vì hàm đã thay đổi giá trị của biến toàn cục

[move/copy question to another bank](#)



Question 2 pts



Cho đoạn mã python sau:

```
def m(a, b=1, c=2):
```

```
    return a + 10*b + 100*c
```

```
print(m(3), m(3, c=0), m(3, 0, 1))
```

Kết quả sau khi chạy đoạn code trên là:

- ☐ 213 13 103
- ☐ 213 3 103
- ☐ 213 13 13
- ☐ 123 13 103
- ☐ 213 3 13

[move/copy question to another bank](#)



Question 2 pts



Cho đặc tả (Specification) của hàm sau đây. Hãy xác định dòng mã nào bị thiếu hoặc sai logic để hàm hoạt động đúng theo đặc tả và đảm bảo tính bền vững?

```
def calculate_average(scores):
```

```
    """
```

```
    Tính điểm trung bình từ danh sách scores.
```

```
    Input: scores (list of numbers)
```

```
    Output: float - điểm trung bình.
```

```
    Raises: ValueError nếu scores trống.
```

```
    """
```

```
    # [VỊ TRÍ CẦN KIỂM TRA]
```

```
    total = sum(scores)
```

```
    return total / len(scores)
```

- ☐ Cần thêm assert scores != [], ""List is empty"" vào vị trí kiểm tra.

- ☐ Cần thêm `if len(scores) == 0: raise ValueError("Empty list")`.
- ☐ Cần thêm `if scores == None: return 0`.
- ☐ Cần sửa `sum(scores)` thành một vòng lặp `for` để kiểm tra từng phần tử.
- ☐ Về mặt cú pháp có thể dùng cả `if` và `assert`, nhưng `assert` phù hợp hơn với yêu cầu Raises ValueError của đặc tả.

[move/copy question to another bank](#)



Question 2 pts



Phân tích đoạn mã dưới đây. Khi thực thi, chương trình sẽ dừng lại (tung ra AssertionError) ở dòng nào?

```
def power(base, exp):
    if exp == 0: return 1
    if exp < 0: return 1 / power(base, -exp)
    return base * power(base, exp - 1)
```

Test cases

```
assert power(2, 3) == 8      # Dòng 1
assert power(5, 0) == 1     # Dòng 2
assert power(2, -1) == 0.5  # Dòng 3
assert power(-2, 2) == -4   # Dòng 4
assert power(10, 2) == 100  # Dòng 5
```

- ☐ Dòng 1
- ☐ Dòng 3
- ☐ Dòng 4
- ☐ Dòng 5
- ☐ Không có dòng nào, tất cả các test đều vượt qua.

[move/copy question to another bank](#)



Question 2 pts



Đặc tả hàm `clamp(x, lo, hi)`:

Trả về `x` nếu `lo <= x <= hi`

Trả về `lo` nếu `x < lo`

Trả về `hi` nếu `x > hi`

Giả sử `lo <= hi` luôn đúng.

Một bạn viết hàm như sau:

```
def clamp(x, lo, hi):
    if x < lo:
        return lo
```

```
if x > hi:
```

```
    return hi
```

```
return hi
```

Bộ test nào dưới đây chắc chắn phát hiện lỗi (tức là làm kết quả sai so với đặc tả)?

- ☐ `clamp(-1, 0, 10) == 0`
- ☐ `clamp(11, 0, 10) == 10`
- ☐ `clamp(0, 0, 10) == 0`
- ☐ `clamp(10, 0, 10) == 10`
- ☐ `clamp(5, 0, 10) == 5`

[move/copy question to another bank](#)



Question 2 pts



Đây là output đoạn mã sau:

```
def writer():
```

```
    title = 'Sir'
```

```
    name = (lambda x:title + ' ' + x)
```

```
    return name
```

```
who = writer()
```

```
print(who('Arthur'))
```

- ☐ Arthur Sir
- ☐ Sir Arthur
- ☐ Arthur
- ☐ Không đáp án đúng

[move/copy question to another bank](#)



Question 2 pts



Yêu cầu: Nhập một chuỗi từ bàn phím, chuyển toàn bộ thành chữ thường, rồi in ra.

Xét đoạn mã sau:

```
text = input
```

```
result = text.lower
```

```
print(result)
```

Kết quả thực thi đoạn mã in ra là gì?

- ☐ In ra chuỗi người dùng nhập ở dạng chữ thường - đúng yêu cầu
- ☐ In ra chuỗi người dùng nhập - sai vì thiếu `.lower()`

- ☐ In ra một object hàm - sai vì không gọi hàm
- ☐ Lỗi TypeError do lower không nhận tham số
- ☐ Không in gì

[move/copy question to another bank](#)



Question 2 pts



Yêu cầu: Tính tổng tiền phải trả, gồm:

price: giá gốc

tax: phần trăm thuế

discount: phần trăm giảm

Công thức:

```
total = price + price * tax / 100 - price * discount / 100
```

Định nghĩa hàm:

```
def calculate_total(price, tax, discount):  
    return price + price * tax / 100 - price * discount / 100
```

Xét đoạn mã sử dụng:

```
total = calculate_total(200, discount=10, tax=5)  
print(total)
```

Kết quả thực thi đoạn mã trên in ra là gì?

- ☐ 195.0 - đúng vì tax và discount đều được áp dụng
- ☐ 210.0 - sai vì tax được áp dụng trước discount
- ☐ 190.0 - đúng với công thức
- ☐ Lỗi TypeError do trộn positional và keyword arguments sai cách
- ☐ 200.0 - công thức bị sai

[move/copy question to another bank](#)



Question 2 pts



Giả sử hàm `calculate_discount(price, member)` được viết để tính giá sau khi giảm (giảm 10% cho thành viên). Dòng assert nào sau đây sẽ gây ra lỗi `AssertionError` với đoạn code hiện tại?

```
def calculate_discount(price, is_member):  
    if is_member:
```

```
final_price = price * 0.9
```

```
else:
```

```
final_price = price
```

```
assert calculate_discount(100, False) == None
```

```
assert calculate_discount(100, False) == 100
```

- ☐ Dòng test số 1
- ☐ Dòng test số 2
- ☐ Không dòng nào gây lỗi
- ☐ Cả hai dòng đều sẽ gây lỗi

[move/copy question to another bank](#)



Question 2 pts



Phát biểu nào mô tả đúng nhất về *specification* (đặc tả) của một hàm?

- ☐ Đặc tả mô tả *cách* cài đặt chi tiết từng dòng lệnh trong hàm.
- ☐

Đặc tả mô tả hàm *làm gì* (hành vi), thường viết dưới dạng docstring: tham số, kết quả trả về, và điều kiện liên quan.

- ☐ Đặc tả chỉ gồm tên hàm và danh sách tham số (không cần mô tả).
- ☐ Đặc tả chỉ cần viết sau khi đã code xong để "cho đẹp".
- ☐ Đặc tả là một file riêng, không liên quan đến docstring.

[move/copy question to another bank](#)



Question 2 pts



Trong file `test_fib.py`, có đoạn:

```
assert fib(10) == 55
```

Ý nghĩa đúng nhất của dòng lệnh này là gì?

- ☐ Luôn in ra 55 nếu fib(10) chạy đúng.
- ☐ Nếu fib(10) khác 55 thì chương trình vẫn chạy bình thường, chỉ cảnh báo.
- ☐ Nếu fib(10) khác 55 thì phát sinh AssertionError (test thất bại).
- ☐ Dòng này làm cho fib(10) luôn trả về 55.
- ☐ Dòng này dùng để đo độ phức tạp thời gian của fib(10).

[move/copy question to another bank](#)



Question 2 pts



Khi một test case bị fail, hành động nào sau đây phù hợp nhất theo quy trình làm việc khoa học?

- ☐ Xóa test case đó vì “khó quá”.
- ☐ Chỉnh sửa ngẫu nhiên nhiều chỗ trong code cho đến khi hết lỗi.
- ☐ Bỏ qua lỗi và tiếp tục viết thêm tính năng khác.
- ☐ Đọc kỹ *error message*, xác định nguyên nhân, sau đó sửa đúng chỗ và chạy lại toàn bộ test.
- ☐ Đổi expected output của test để test “pass”.

[move/copy question to another bank](#)



Question 2 pts



Cho định nghĩa hàm sau:

```
def check_value(n):  
    """Kiểm tra số n.  
    Trả về True nếu n là số dương, ngược lại False."""  
    return n > 0
```

Khẳng định nào sau đây về việc kiểm thử hàm này là đúng?

- ☐ assert check_value(5) == True sẽ không gây ra lỗi
- ☐ assert check_value(-1) == True sẽ không gây ra lỗi
- ☐ Hàm trên sẽ gây lỗi AssertionError nếu ta truyền vào số 0
- ☐ Docstring của hàm không cung cấp thông tin về giá trị trả về
- ☐ Câu lệnh assert check_value(10) sẽ trả về kết quả là None ra màn hình

[move/copy question to another bank](#)



Question 2 pts



Cho đoạn mã python sau:

```
a = [5, 1, 4, 2]  
for i in range(len(a) - 1):  
    if a[i] > a[i+1]:  
        a[i], a[i+1] = a[i+1], a[i]  
print(a)
```

Kết quả sau khi chạy đoạn code trên là:

- ☐ [1, 2, 4, 5]
- ☐ [1, 4, 2, 5]
- ☐ [5, 1, 2, 4]
- ☐ [1, 5, 4, 2]

☐ [5, 4, 2, 1]

[move/copy question to another bank](#)